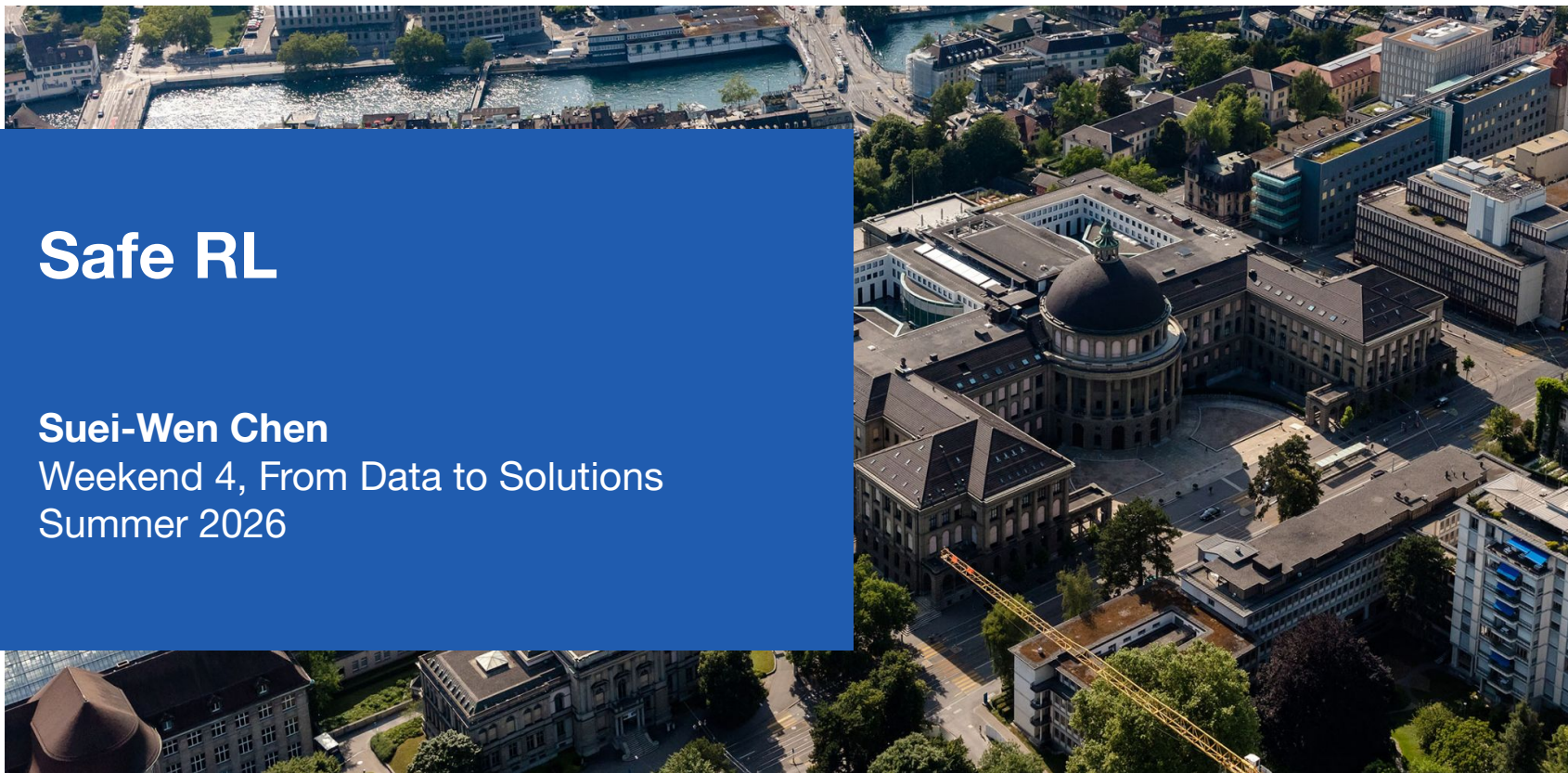


Safe RL

Suei-Wen Chen

Weekend 4, From Data to Solutions
Summer 2026



Roadmap of This Weekend

- 1 Introduction
- 2 Policy Gradient Methods: REINFORCE & Baselines
- 3 Variance Reduction: Actor-Critic
- 4 Credit Assignment: GAE and PPO
- 5 Human Alignment & Reasoning: RL in LLM
- 6 New Paradigm: Safe/Risk-Aware RL
- 7 Project Release



Standard RL & Where It Falls Short

So far, RL = Learn to achieve the goal by **maximizing the average return**

What if our RL agent finds **undesirable ways** to achieve higher payoff?

 High-frequency trading bot learns to use **extreme leverages**

- Massive typical gains outweigh small probability of ruin

 Autonomous car **aggressively change lanes** to reduce travel time

- Saved time outweighs small probability of crashing

 Recommendation algorithm promotes **harmful content** to increase retention

- (Maybe) this was not part of the reward design

Can't we just add penalization in the reward design?

- Too little penalty gets washed out by low probability & other good behavior
- Too much penalty can lead to reward hacking / unwanted side effects
 - Autonomous car does not move to avoid large penalty
- Reward in expectation cannot enforce hard constraints
 - Minimum distance to the vehicle ahead
 - No violent content can be recommended to underage users
- Goal-constraint trade-offs are usually nonlinear
 - Allow for a certain violation budget
 - Maximize the profit for the worst 0.01% scenarios

What is safety?

Formalizations:

- **State constraints:** No undesirable state is ever visited
- **Budget constraints:** Keep expected safety cost below a threshold
- **Robustness:** Maximize the worst-case return

Where safety is enforced:

- **Safe deployment:** Violations are tolerated during training (e.g., simulation), but prohibited at test time
- **Safe exploration:** Violations must be avoided during training itself (e.g., learning directly from interacting with the real world)

Multiple ways to be safe or risk-sensitive

1. Constrained Markov Decision Processes (CMDPs)

Model a **cost function** $c(s, a)$ in addition to the reward $r(s, a)$

2. Shielded Reinforcement Learning

Override unsafe actions with minimal intervention

3. Distributional Reinforcement Learning

Track **entire return/risk distributions** to perform risk-sensitive optimization

1. Constrained Markov Decision Processes (CMDPs)

Each state-action pair $(\mathbf{s}, \mathbf{a})$ is associated with:

- immediate reward $\mathbf{r}(\mathbf{s}, \mathbf{a})$
- cost function(s) $\mathbf{c}(\mathbf{s}, \mathbf{a})$

Goal: Find policy π_{θ} maximizing expected return

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_t \gamma^t r(S_t, A_t) \right]$$

while the expected cost stays within a budget

$$J_c(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_t \gamma^t c(S_t, A_t) \right] \leq d$$

Example: Adaptive drug dosages for chronic conditions

state = patient condition action = dosage

reward = treatment efficacy

cost = produced toxicity level, radiation exposure, ...

How to solve CMDP tasks?

[Constrained Policy Optimization \(2017\)](#): Trust-Region Policy Search

Maximizes reward while bounding policy shift via KL-divergence.

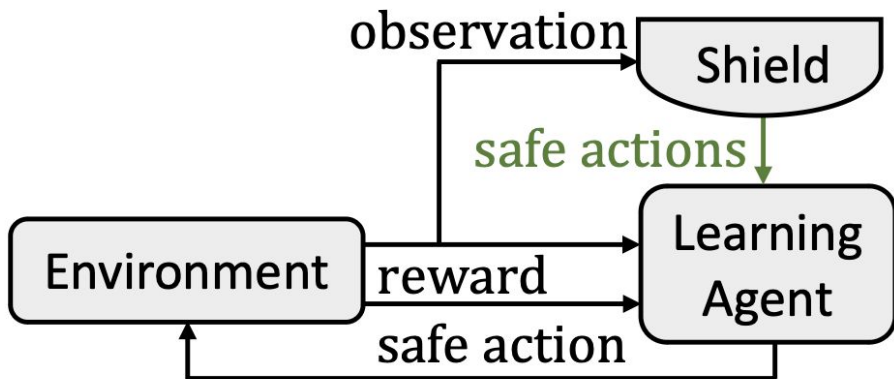
 Safety Guarantees: Each policy update respects cost limits

 Computationally expensive: second-order optimization

In practice, first-order methods like PPO-Lagrangian are used for scale and speed

2. Shielding: Enforce safety with minimal interference

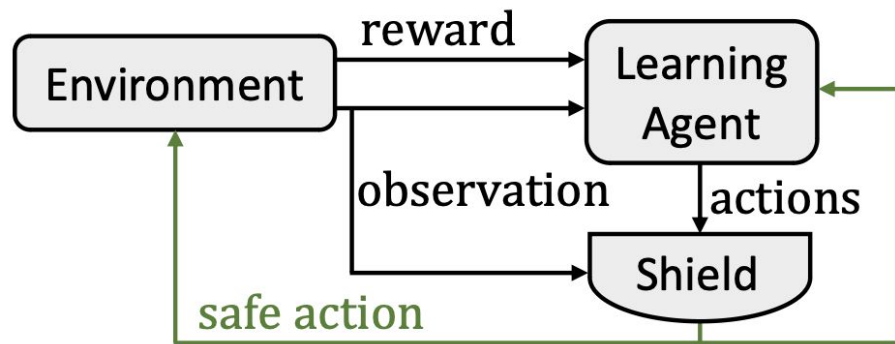
Preemptive Shielding



At step t (current state = s_t):

- Shield provides a set of safe actions at s_t
- Agent chooses action a_t from this set

Post-Posed Shielding



At step t (current state = s_t):

- Agent chooses action a_t
- Shield checks whether a_t is safe
- If unsafe, replace a_t with a safe action

Preemptive Shielding

Implementation:

- Compute all safe actions $\mathbf{A}_{\text{safe}}(\mathbf{s}_t)$
- For every unsafe action $\mathbf{a}$, set $\pi(\mathbf{a} | \mathbf{s}) = 0$ (e.g. force logit or $Q(\mathbf{s}, \mathbf{a})$ to $-\infty$)
- Pick the next action using π

Learning/Training:

- The shield changes the original MDP to another (safe) MDP
- Apply any standard (unconstrained) RL algorithm to this safe MDP

Post-posed Shielding

Hard Constraints can be enforced by:

- A predefined backup safe policy
- Control Barrier Functions: Project $\mathbf{a}_t$ to the closest safe action

Soft Constraints can be enforced by penalizing deviation from a safe policy:

- At each step, find policy π maximizing

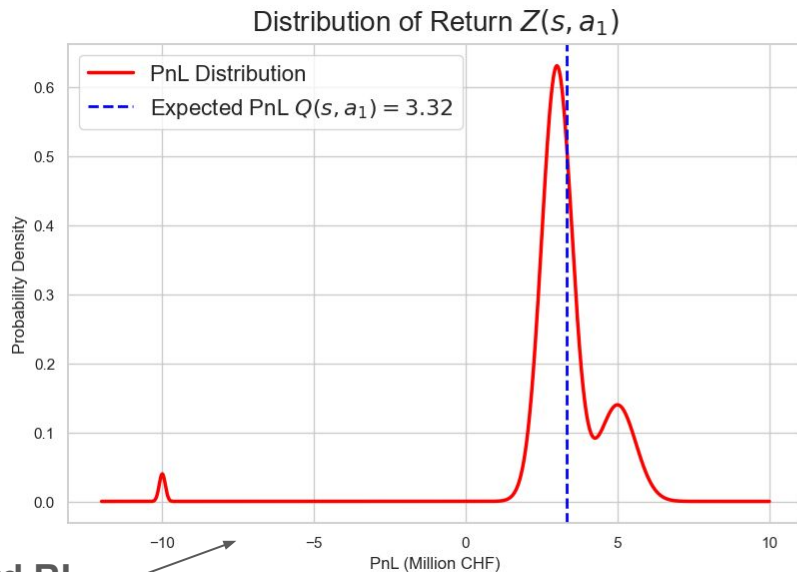
$$\mathbb{E}_{a \sim \pi} [Q_{\theta}(s, a)] - \beta \cdot D_{\text{KL}} \left(\pi(\cdot | s) \parallel \pi_{\text{safe}}(\cdot | s) \right)$$

comparison baseline

(e.g., expert demonstration, $\text{Softmax}(\text{Masked}(Q))$, uniform on $\mathbf{A}_{\text{safe}}(s)$)

3. Distributional Reinforcement Learning ([2017](#))

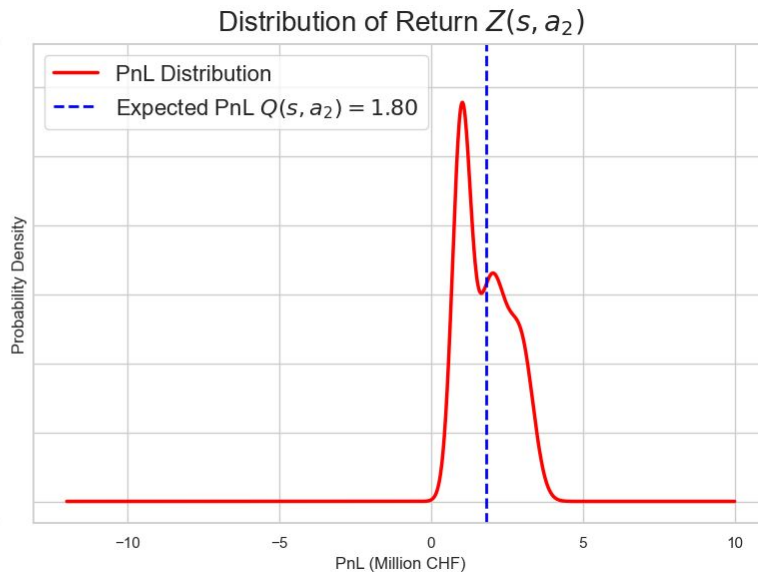
Two trading options $\mathbf{a}_1$ and $\mathbf{a}_2$ at the current state $\mathbf{s}$ (market + portfolio):



Standard RL

picks this action

Higher average return
but 1% chance of ~10M loss



Lower average return
but consistent gain

Risk Measures: Quantify the riskiness of a distribution

A risk measure models **how risky a random variable (or its distribution) is**.

Usually in RL, **risk = left tail of return or right tail of cost**

Common Risk Measures:

Value-At-Risk (VaR) at level α :

- The outcome threshold defining the worst α -fraction of cases (the α -quantile)

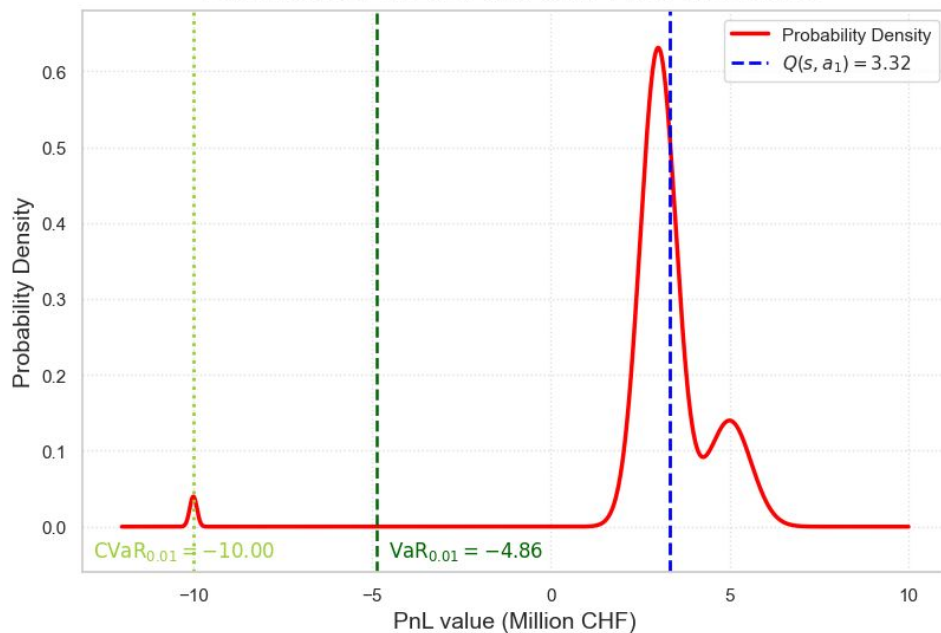
Conditional Value-At-Risk (CVaR) at level α : (standard in safe RL)

- The average outcome in the worst α -fraction of cases
- also called Average Value-At-Risk (AVaR) or Expected Shortfall (ES)*

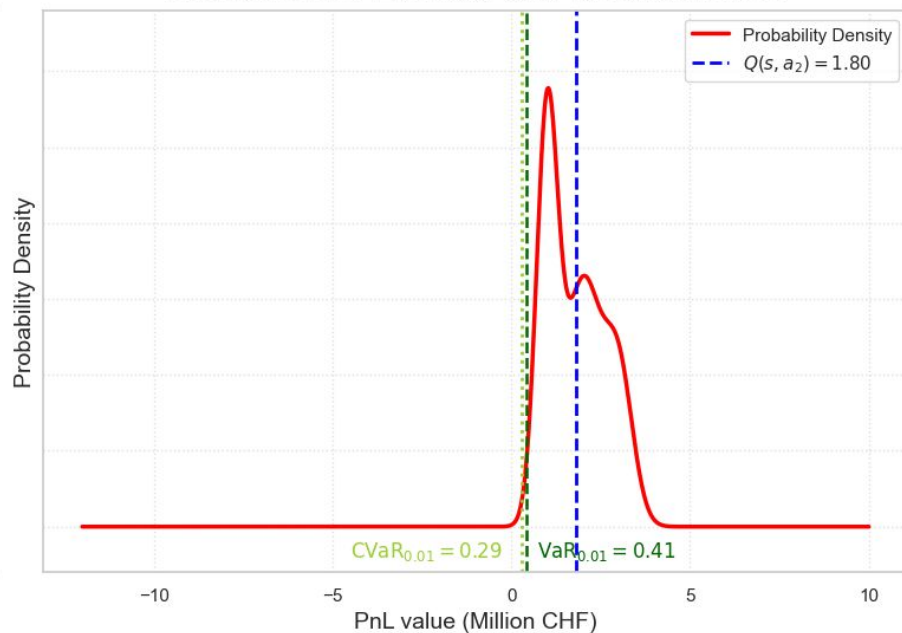
Comparison between VaR and CVaR (pdf view)

CVaR “goes into the tail” while VaR does not

Distribution of $Z(s, a_1)$ with Risk Measures

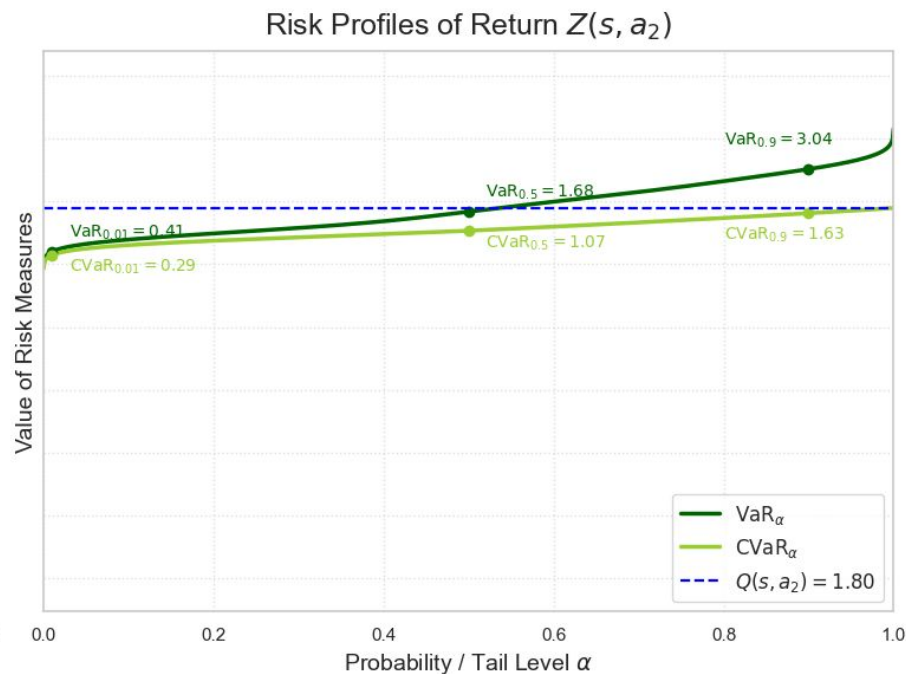
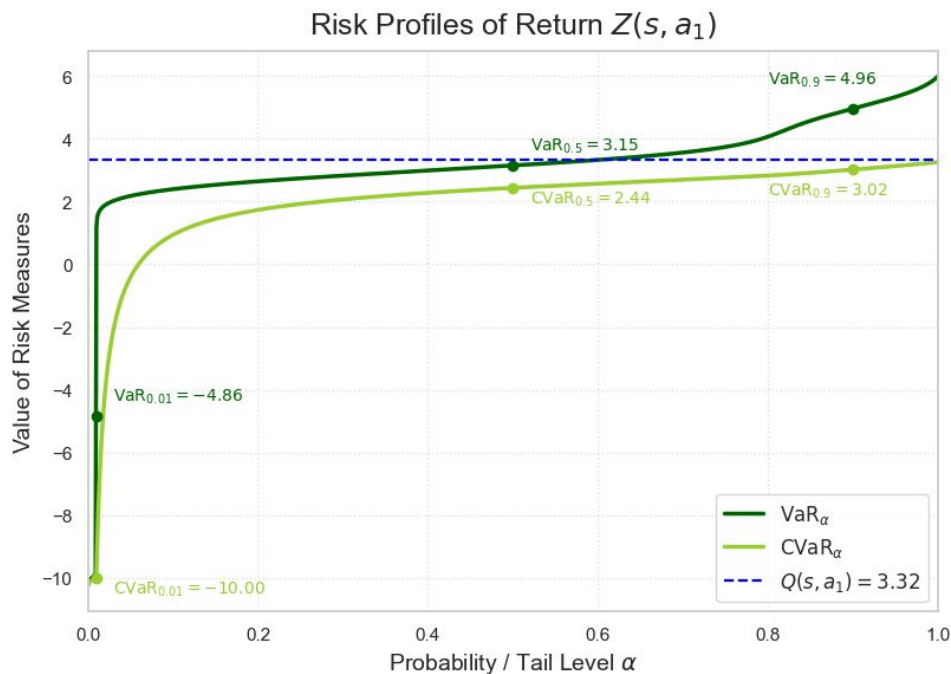


Distribution of $Z(s, a_2)$ with Risk Measures



Comparison between VaR and CVaR (quantile view)

CVaR = Average of VaR below the threshold



Risk-Sensitive Optimization

Standard RL finds policy π which maximizes the average return

$$V^\pi(s) = Q^\pi(s, \pi(s)) = \mathbb{E}[Z^\pi(s, \pi(s))]$$

Distributional RL can achieve robustness by **optimizing some risk measure**, e.g.,

$$\max_{\pi} \text{CVaR}_{\alpha}(Z^\pi(s, \pi(s)))$$

or by **maximizing the average return with risk constraints**, e.g.,

$$\max_{\pi} \mathbb{E}[Z^\pi(s, \pi(s))] \quad \text{subject to} \quad \text{CVaR}_{\alpha}(Z^\pi(s, \pi(s))) \geq d$$

How to conduct learning in the distributional setting

Many concepts in standard RL can be extended to be distributional

- Bellman Equations, TD-Learning, Policy Gradients, Actor-Critic, ...

A lot of theoretical/computational complications:

- **Optimal policies** no longer deterministic/stationary/Markov
- Tricky to learn **full distributions** (representation, sample complexity)
- Risk-sensitive objectives require **extra care to optimize**

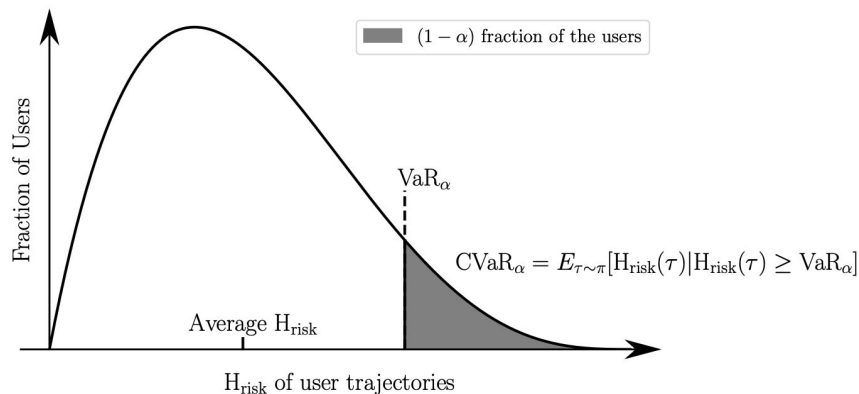
Application: Safe Recommendation System

Goal: Recommend movies to users without too much violence exposure

Reward: $r_{\text{user, movie}} = \text{movie_quality} \times \text{user_genre_affinity}$

Risk: Each movie is associated with a violence score

H_{risk} = average violence score of movies recommended to each user

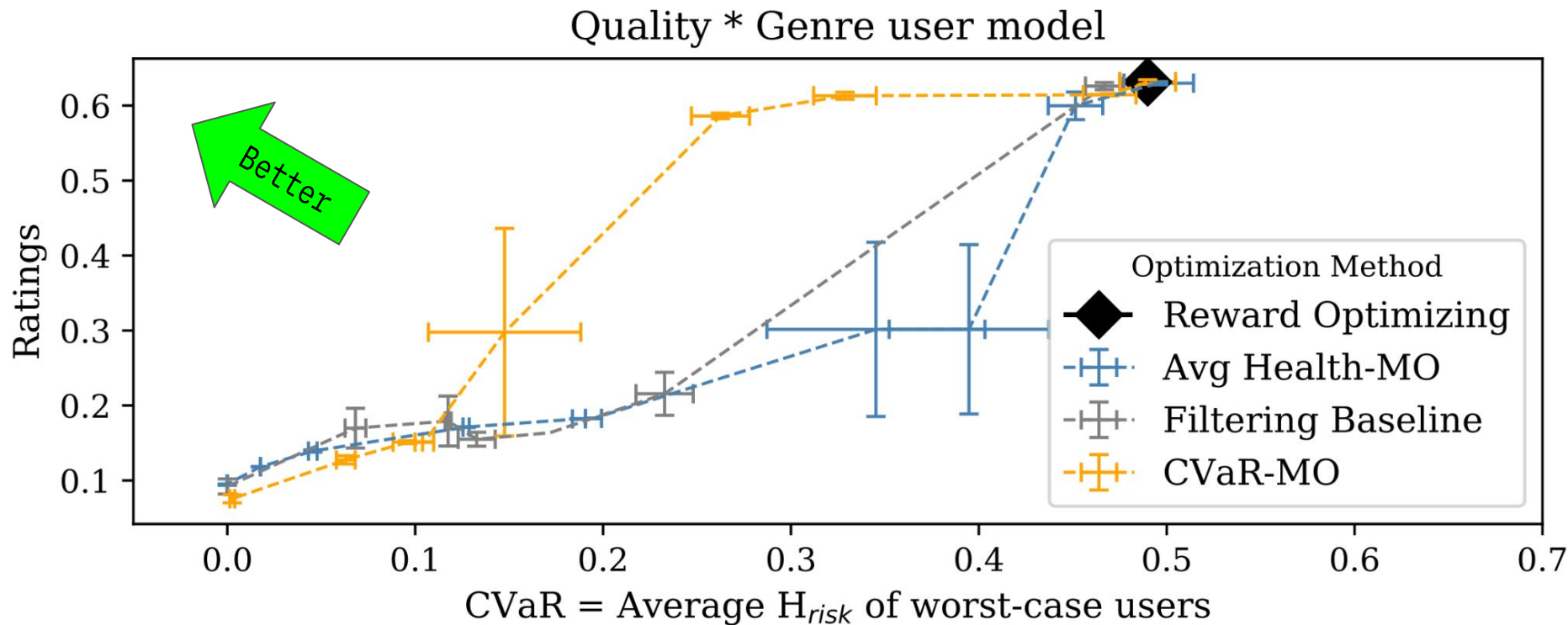


$$\pi_{\lambda}^{\theta*} = \operatorname{argmax}_{\theta} \quad \mathbb{E}_{u \sim \mathcal{U}, \tau \sim \pi_{\theta}} [R(\tau)] - \lambda \mathbb{E}_{\tau \sim \pi_{\theta}} [H_{\text{risk}}(\tau)]$$

$$\pi_{\alpha, \lambda}^{\theta*} = \operatorname{argmax}_{\theta} \quad \mathbb{E}_{u \sim \mathcal{U}, \tau \sim \pi_{\theta}} [R(\tau)] + \lambda \text{CVaR}_{\alpha}(H_{\text{risk}} | \pi)$$

How much you care about the risk

Application: Safe Recommendation System



Summary

Paradigm	Safety Formalization	Intuition
Constrained MDPs	Cumulative budget in expectation	Don't spend more than the safety budget on average
Shielded RL	Restricting unsafe actions per state	Never take actions leading into dangerous territories
Distributional RL	Risk measure over return/cost distribution	Protect against the worst-case outcomes